



HỆ ĐIỀU HÀNH

CHƯƠNG 4: ĐỊNH THỜI CPU (PHẦN 2)

Định thời CPU là hoạt động quan trọng của thành phần quản lý tiến trình và có ảnh hưởng rất lớn đến hiệu suất máy tính cũng như trải nghiệm của người dùng. Trong chương này, người học được trình bày về mục đích và các tiêu chuẩn định thời, cũng như các chiến lược định thời CPU cơ bản.



NỘI DUNG

4. Các giải thuật định thời

- Highest Response Ratio Next (HRRN)
- Multilevel Queue
- Multilevel Feedback Queue

5. Định thời tiểu trình (Thread scheduling)

6. Định thời đa bộ xử lý (Multiple-processor scheduling)

7. Định thời theo thời gian thực (Real-time CPU scheduling)

8. Định thời trên một số hệ điều hành

- Linux
- Windows
- Solaris



CÁC GIẢI THUẬT ĐỊNH THỜI

4.6. Highest Response Ratio Next (HRRN)

04.



4.6. Highest Response Ratio Next (HRRN)

- Chọn tiến trình kế tiếp có giá trị **RR (Response ratio)** lớn nhất.
- Các tiến trình ngắn được ưu tiên hơn (vì *service time* nhỏ).

$$RR = \frac{\text{time spent waiting} + \text{expected service time}}{\text{expected service time}}$$

- Câu hỏi thảo luận:
 - So sánh với cơ chế Aging?
 - Ưu điểm và hạn chế của hướng tiếp cận này?



CÁC GIẢI THUẬT ĐỊNH THỜI

4.7. Multilevel Queue

04.



4.7. Multilevel Queue

- *Ready queue* được chia thành nhiều hàng đợi riêng biệt theo một số tiêu chuẩn sau:
 - Đặc điểm và yêu cầu định thời của tiến trình.
 - Phân loại tiến trình: Foreground (interactive) và background,...
- Tiến trình được gán cố định vào một hàng đợi, mỗi hàng đợi sử dụng giải thuật định thời riêng.



4.7. Multilevel Queue

Việc định thời giữa các hàng đợi

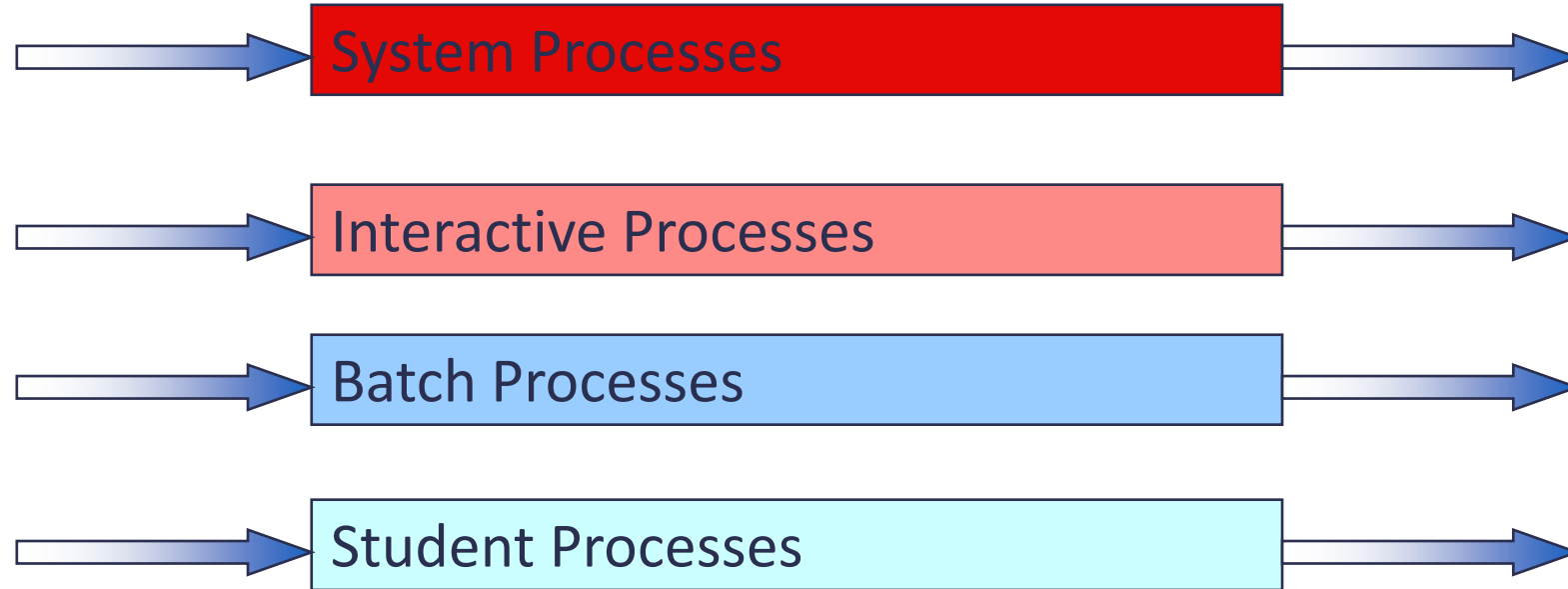
- Hệ điều hành cần phải định thời cho các hàng đợi:
 - **Fixed priority scheduling**: phục vụ từ hàng đợi có độ ưu tiên cao đến thấp → Có thể phát sinh vấn đề đói tài nguyên (**starvation**).
 - **Time slice**: mỗi hàng đợi được nhận một khoảng thời gian chiếm CPU và phân phối cho các tiến trình trong hàng đợi khoảng thời gian đó. Ví dụ: 80% cho hàng đợi foreground định thời bằng RR và 20% cho hàng đợi background định thời bằng giải thuật FCFS.



4.7. Multilevel Queue

Ví dụ phân chia hàng đợi và tiến trình

Độ ưu tiên cao nhất



Độ ưu tiên thấp nhất

4.7. Multilevel Queue

Hạn chế của Multilevel Queue

- Tiến trình không thể chuyển từ hàng đợi này sang hàng đợi khác:
 - Có thể làm giảm hiệu suất hệ thống trong trường hợp một hàng đợi có nhiều tiến trình trong khi các hàng đợi khác lại trống.
 - Khắc phục bằng cơ chế feedback: cho phép tiến trình di chuyển một cách thích hợp giữa các hàng đợi khác nhau.
 - Giải thuật: **Multilevel Feedback Queue**



CÁC GIẢI THUẬT ĐỊNH THỜI

4.8. Multilevel Feedback Queue

04.



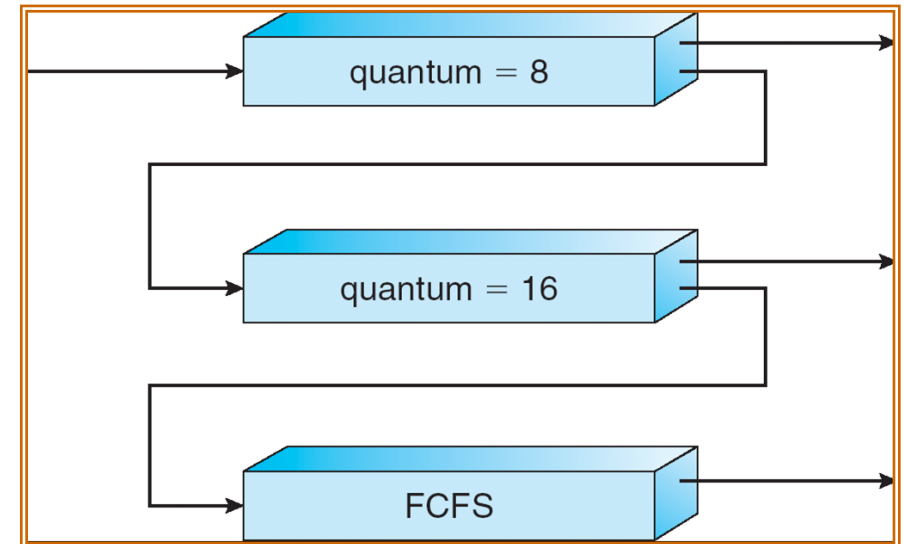
4.8. Multilevel Feedback Queue

- Phân loại tiến trình dựa trên các đặc tính về **CPU-burst**.
- Sử dụng chế độ **trưng dụng** (preemptive).
- Sau một khoảng thời gian nào đó, các tiến trình hướng I/O và tiến trình interactive sẽ ở các hàng đợi có độ ưu tiên cao hơn còn các tiến trình hướng CPU sẽ ở các hàng đợi có độ ưu tiên thấp hơn.
- Một tiến trình đã chờ quá lâu ở một hàng đợi có độ ưu tiên thấp có thể được chuyển đến hàng đợi có độ ưu tiên cao hơn (cơ chế aging).

4.8. Multilevel Feedback Queue

Ví dụ về Multilevel Feedback Queue

- Ví dụ: Có 3 hàng đợi
 - Q0, dùng RR với quantum 8 ms
 - Q1, dùng RR với quantum 16 ms
 - Q2, dùng FCFS





4.8. Multilevel Feedback Queue

Các vấn đề với Multilevel Feedback Queue

- Định thời dùng multilevel feedback queue đòi hỏi phải giải quyết các vấn đề sau:
 - Số lượng hàng đợi bao nhiêu là thích hợp?
 - Dùng giải thuật định thời nào ở mỗi hàng đợi?
 - Làm sao để xác định thời điểm cần chuyển một tiến trình đến hàng đợi cao hơn hoặc thấp hơn?
 - Khi tiến trình yêu cầu được xử lý thì đưa vào hàng đợi nào là hợp lý nhất?



CÁC GIẢI THUẬT ĐỊNH THỜI

4.9. So sánh các giải thuật

04.



4.9. So sánh các giải thuật

- Giải thuật định thời nào là tốt nhất?
- Câu trả lời phụ thuộc các yếu tố sau:
 - Tần suất tải việc (System workload).
 - Sự hỗ trợ của phần cứng đối với dispatcher.
 - Sự tương quan về trọng số của các tiêu chuẩn định thời như response time, hiệu suất CPU, throughput,...
 - Phương pháp định lượng so sánh.



ĐỊNH THỜI TIỂU TRÌNH

05.



5. Định thời tiến trình

- Trên các hệ điều hành hiện đại có hỗ trợ tiến trình, **tiến trình được định thời**, không phải tiến trình.
- Có sự phân biệt giữa tiến trình người dùng và tiến trình hạt nhân khi định thời:
- **Tiến trình người dùng (user thread)** được định thời thông qua các thư viện quản lý tiến trình:
 - Phạm vi định thời là bên trong tiến trình (process-contention scope - PCS).
 - Thường được thực hiện bằng cách thiết lập độ ưu tiên (bởi người lập trình).
- **Tiến trình hạt nhân (kernel thread)** được định thời trên tất cả các CPU khả dụng. Phạm vi định thời là toàn hệ thống (system-contention scope - SCS).



ĐỊNH THỜI ĐA BỘ XỬ LÝ

06.



6. Định thời đa bộ xử lý

- Định thời CPU trở nên phức tạp hơn khi hệ thống có nhiều bộ xử lý.
- Khái niệm đa bộ xử lý có thể là một trong các dạng sau:
 - CPU có nhiều lõi vật lý (Multicore CPUs)
 - CPU có nhiều luồng xử lý trên một lõi (Multithreaded cores)
 - Hệ thống NUMA (non-uniform memory access)
 - Đa xử lý không đồng nhất (Heterogeneous multiprocessing)
- Có hai cách tiếp cận phổ biến: **đa xử lý bất đối xứng** (asymmetric multiprocessing) và **đa xử lý đối xứng** (symmetric multiprocessing - SMP).



ĐỊNH THỜI ĐA BỘ XỬ LÝ

6.1. Đa xử lý bất đối xứng

06.



6.1. Đa xử lý bất đối xứng

- Tất cả các thao tác lập lịch, xử lý I/O được thực hiện bởi một bộ xử lý – master server.
- Các bộ xử lý còn lại chỉ thực thi user code.
- Ưu điểm: đơn giản, chỉ một bộ xử lý truy xuất dữ liệu hệ thống, không cần chia sẻ dữ liệu.
- Nhược điểm: master server có thể bị nghẽn cổ chai (bottleneck), làm giảm hiệu năng của hệ thống.



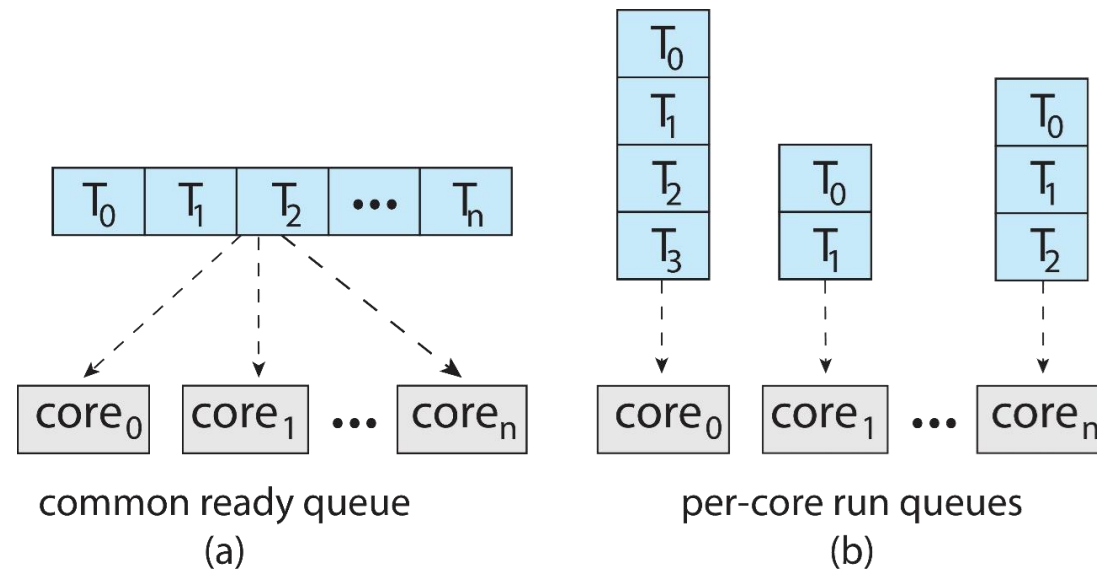
ĐỊNH THỜI ĐA BỘ XỬ LÝ

6.2. Đa xử lý đối xứng

06.

6.2. Đa xử lý đối xứng

- Mỗi bộ xử lý tự định thời cho chính nó.
- Hai hướng tiếp cận để tổ chức các tiểu trình cần định thời:
 - Tất cả tiểu trình nằm trong cùng một *ready queue* (a)
 - Mỗi bộ xử lý tự tổ chức hàng đợi của riêng nó (b)



6.2. Đa xử lý đối xứng

- Tất cả tiểu trình nằm trong cùng một *ready queue*:
 - Tiểu trình có thể không được bộ xử lý nào chọn ?
 - Xuất hiện vùng tranh chấp: Nhiều bộ xử lý có thể chọn định thời cùng một tiểu trình => Cần có cơ chế kiểm tra và khóa (lock) việc truy xuất tiểu trình => Hiệu năng hệ thống có thể giảm do nghẽn cổ chai.
- Mỗi bộ xử lý tự tổ chức hàng đợi của riêng nó:
 - Hiệu năng không bị ảnh hưởng do các vấn đề khi dùng chung một hàng đợi => Hướng tiếp cận phổ biến trên các hệ thống SMP.
 - Vấn đề: Khối lượng công việc của các bộ xử lý khác nhau?

6.2. Đa xử lý đối xứng

Cân bằng tải (Load balancing)

- Một bộ xử lý có quá nhiều tải, trong khi các bộ xử lý khác rỗi => Cần đảm bảo các bộ xử lý đều được sử dụng hiệu quả.
- Mục tiêu của cân bằng tải là phân phối khối lượng công việc (workload) đều nhau cho các CPU.
- Có hai cách cân bằng tải:
 - **Push migration:** Một tác vụ đặc biệt sẽ kiểm tra định kỳ tải của từng CPU. Nếu tình trạng quá tải xuất hiện, hệ thống sẽ di chuyển (đẩy) tác vụ từ CPU bị quá tải sang các CPU khác.
 - **Pull migration:** CPU rỗi kéo (pull) tác vụ đang chờ từ CPU bận.

6.2. Đa xử lý đối xứng

Processor affinity

- Khi một tác vụ chạy trên một bộ xử lý, bộ nhớ đệm (cache) của bộ xử lý đó lưu trữ dữ liệu được truy xuất bởi tác vụ => tác vụ có tính liên kết với bộ xử lý (processor affinity).
- Cân bằng tải sẽ ảnh hưởng đến processor affinity, cụ thể là khi một tác vụ được dời sang bộ xử lý khác:
 - Cache của bộ xử lý mới phải nạp lại (repopulate).
 - Cache của bộ xử lý cũ phải được giải phóng (invalidate).

→ Phí tổn



6.2. Đa xử lý đối xứng

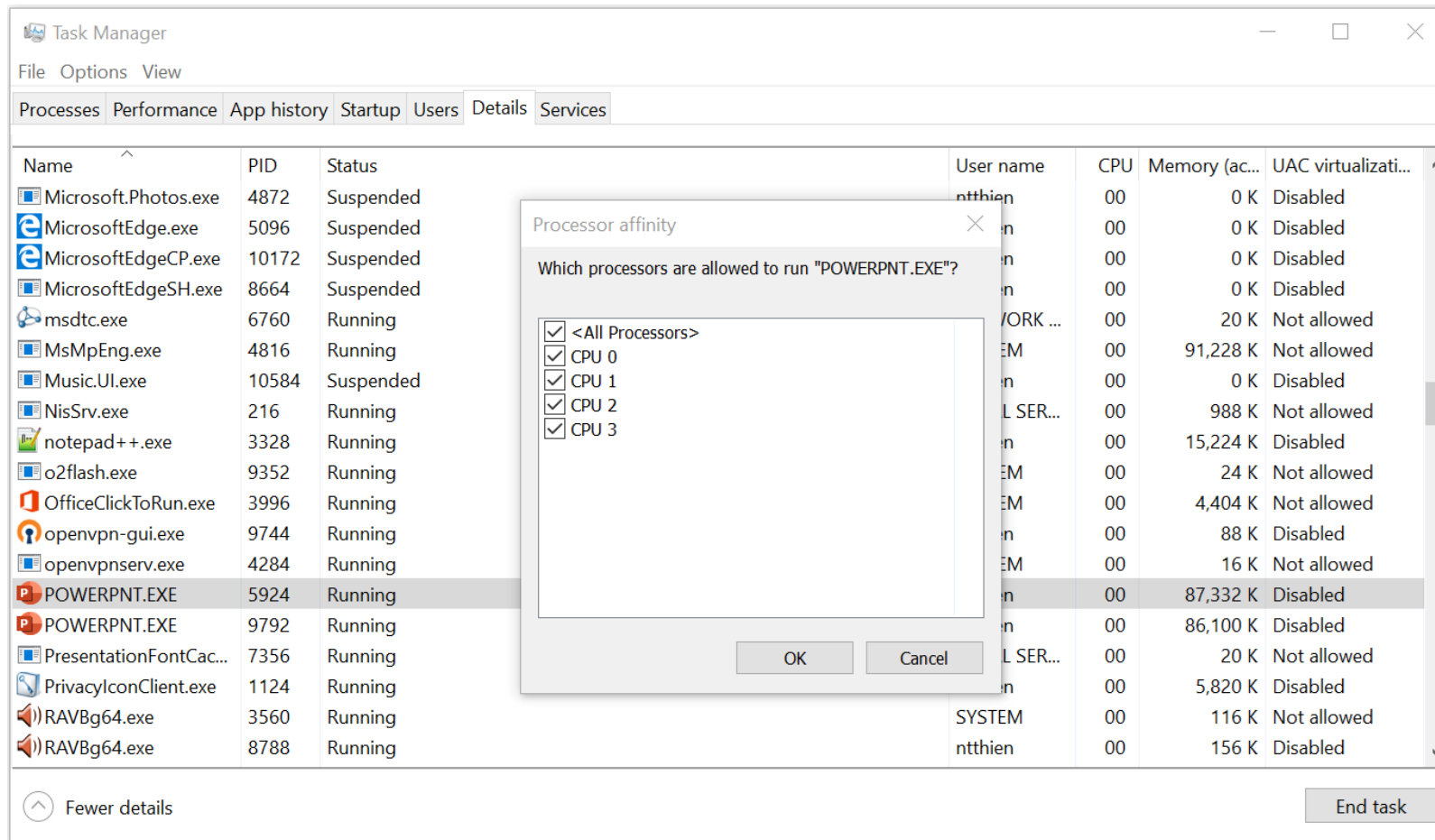
Processor affinity

- Có 2 dạng liên kết:
 - **Liên kết mềm (Soft affinity):** Hệ thống sẽ cố giữ tác vụ chỉ chạy trên bộ xử lý đó (nhưng không đảm bảo).
 - **Liên kết cứng (Hard affinity):** Cho phép tiến trình chọn một tập các bộ xử lý mà nó có thể chạy trên đó.



6.2. Đa xử lý đối xứng

Processor affinity





ĐỊNH THỜI THEO THỜI GIAN THỰC (ĐỌC THÊM)

07.



7. Định thời theo thời gian thực

- Có nhiều thách thức do yêu cầu về tính chất thời gian thực.
- Có 2 dạng hệ thống thời gian thực:
 - **Soft real-time systems:** Các tác vụ quan trọng sẽ được cấp độ ưu tiên lớn nhất, nhưng không đảm bảo bất cứ điều gì khác.
 - **Hard real-time systems:** Tác vụ phải hoàn thành trong deadline của nó.



7. Định thời theo thời gian thực

Định thời theo thời gian thực dựa trên độ ưu tiên

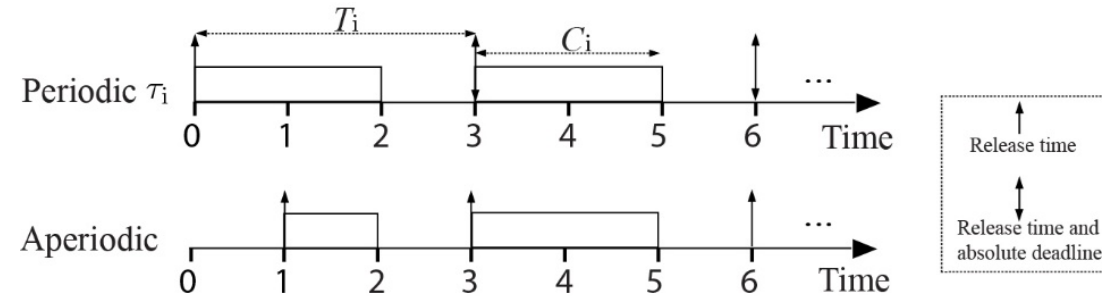
- Hệ thống thời gian thực phải phản hồi ngay lập tức yêu cầu CPU của một tiến trình => Bộ định thời phải hỗ trợ định thời theo độ ưu tiên với chế độ trưng dụng.
- Tiến trình có thêm một đặc trưng mới: tính chu kỳ - yêu cầu CPU trong một khoảng thời gian cố định.
- Khi một tiến trình có chu kỳ yêu cầu CPU, nó có thời gian xử lý C , thời gian deadline d (thời gian nó sẽ được phục vụ bởi CPU) và thời gian chu kỳ T .
 - $0 \leq C \leq d \leq T$

7. Định thời theo thời gian thực

Mô hình tác vụ trong thời gian thực

Periodic tasks

- n periodic tasks;
- Task: $\tau_i = (\mu_i, T_i, C_i)$ ($0 \leq i < n$)
 - μ_i : task utilization,
 - $\sum_{i=0}^{n-1} \mu_i \leq m$ (number of processors)
 - T_i : task period
 - $C_i = \mu_i * T_i$: worst-case execution time
- Job release times and absolute deadlines are predictable.



Aperiodic tasks

- Entering times are unknown a priori
- Soft real-time aperiodic tasks
- No deadline constraints
- Responsiveness is targeted.

Self-Study

Sporadic tasks



7. Định thời theo thời gian thực

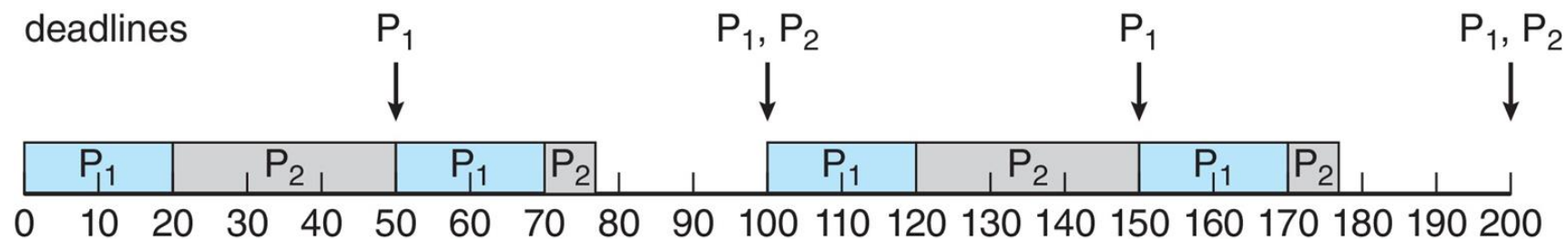
Các giải thuật định thời

- Tác vụ chu kỳ - Periodic Tasks
 - Rate Monotonic - RM
 - Earliest Deadline First - EDF
- Tác vụ phi chu kỳ - Aperiodic Task
 - Total Bandwidth Server – TBS
 - Enhanced Virtual Release Advancing TBS
 - Constant Bandwidth Server

7. Định thời theo thời gian thực

Định thời Rate Monotonic

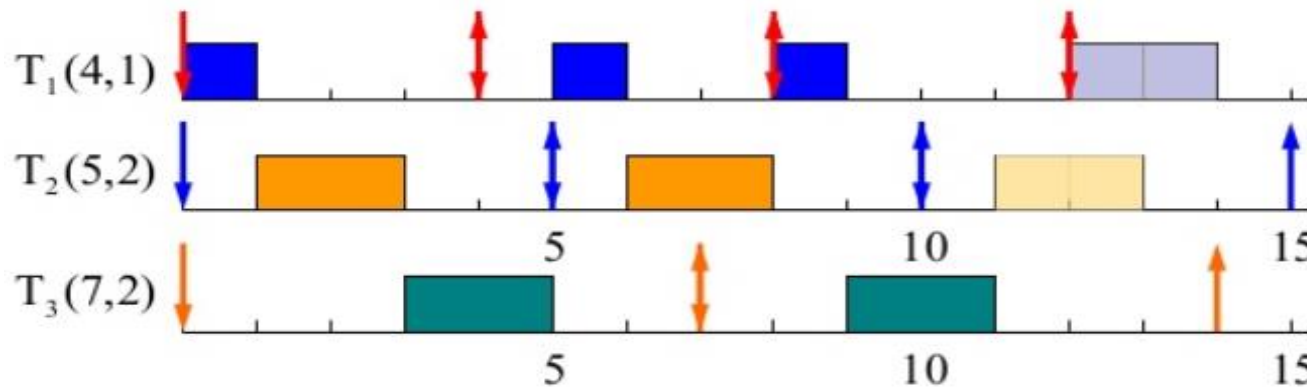
- Độ ưu tiên được gán dựa trên nghịch đảo của chu kỳ => **Chu kỳ ngắn thì độ ưu tiên cao và ngược lại.**
- Tần suất của tác vụ là $1/p$.
- P1 được gán độ ưu tiên cao hơn P2.



7. Định thời theo thời gian thực

Định thời EDF

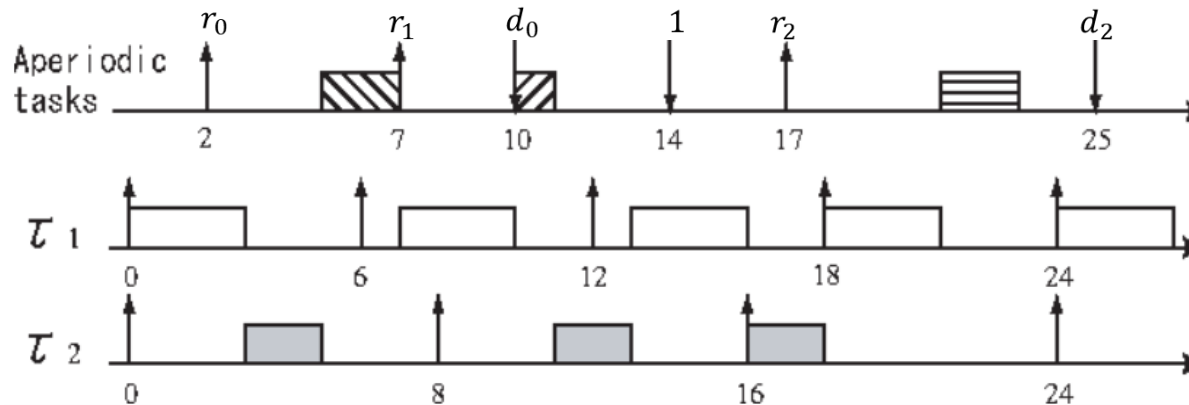
- Tổng hệ số sử dụng CPU (Utilization bound): $U = \sum_{i=1}^n \mu_i \leq 1$
- Thời hạn tuyệt đối (Absolute deadline): $d_i = r_i + T_i$
- Tác vụ có thời hạn hoàn thành nhỏ hơn sẽ có độ ưu tiên lớn hơn.



7. Định thời theo thời gian thực

Định thời Total Bandwidth Server (TBS)

- Dựa trên EDF, định thời tác vụ phi chu kỳ.
- Hệ số sử dụng CPU của tác vụ có chu kỳ: $U_p = \sum_{i=0}^{n_p} \mu_i^p \leq 1$
- Hệ số sử dụng CPU của tác vụ phi chu kỳ: $U_a = 1 - U_p$
- Tác vụ phi chu kỳ: $d_j = r_j + \frac{C_j^{wcet}}{U_a}$





7. Định thời theo thời gian thực

Các giải thuật định thời khác

- Partition Scheduling
 - Partition EDF
- Semi-Partition Scheduling
 - Semi Partition Reservation
- Global Scheduling
 - Global EDF
 - PFAIR – Proportionate Fairness
 - RUN – Reduce to Uniprocessor
 - Local Assignment Algorithm



ĐỊNH THỜI TRÊN MỘT SỐ HỆ ĐIỀU HÀNH

8.1. Định thời trên Linux

08.



8.1. Định thời trên Linux

- Nhân Linux 2.5 trở về trước sử dụng các phiên bản định thời UNIX tiêu chuẩn:
 - Không hỗ trợ tốt các hệ thống nhiều bộ xử lý.
 - Hiệu năng kém nếu có số lượng lớn các tiến trình trong hệ thống.
- Nhân Linux 2.5 sử dụng bộ định thời $O(1)$:
 - Chạy với thời gian hằng số.
 - Định thời theo độ ưu tiên với chế độ trưng dụng.
 - Có hai khoảng ưu tiên: time-sharing và real-time.
 - Giá trị số nhỏ hơn biểu diễn độ ưu tiên lớn hơn.
 - Hoạt động tốt với các hệ thống SMP nhưng đáp ứng kém với các tiến trình interactive.



8.1. Định thời trên Linux

Bộ định thời CFS

- Nhân Linux từ 2.6.23 sử dụng bộ định thời CFS (Completely Fair Scheduler)
 - Định thời theo lớp:
 - Mỗi lớp được gán một độ ưu tiên cụ thể.
 - Bộ định thời chọn tác vụ có độ ưu tiên cao nhất trong lớp có độ ưu tiên cao nhất.
 - Thời gian sử dụng CPU của mỗi tác vụ không dựa trên quantum time cố định mà dựa trên tỷ lệ giờ CPU.
 - Nhân Linux cài đặt sẵn 2 lớp: default và real-time. Các lớp khác có thể được thêm vào.



8.1. Định thời trên Linux

Bộ định thời CFS

- Thời gian sử dụng CPU:
 - Được tính dựa trên giá trị nice được gán cho mỗi tác vụ, có giá trị từ -20 đến 19.
 - Giá trị thấp hơn có độ ưu tiên cao hơn.
 - Target latency – khoảng thời gian mà một tiến trình cần được chạy ít nhất một lần.
 - Target latency có thể tăng lên nếu số lượng tiến trình tăng lên.



8.1. Định thời trên Linux

Bộ định thời CFS

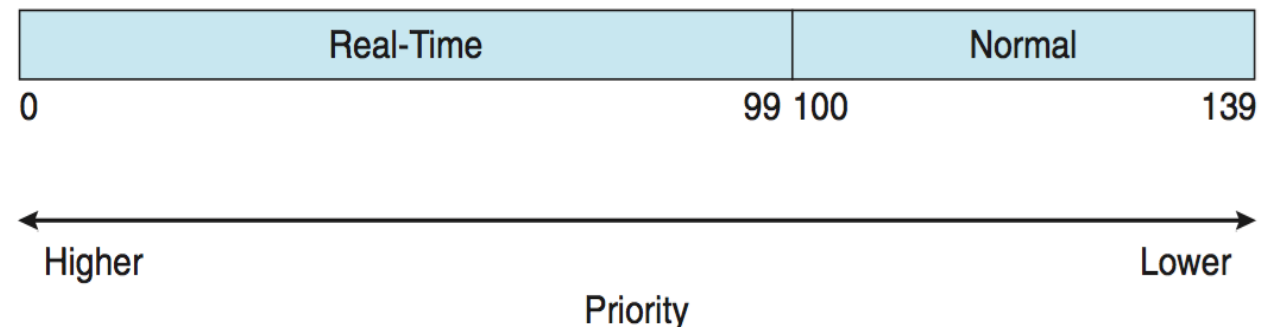
- CFS xác định tác vụ được thực thi kế tiếp qua virtual run time:
 - Mỗi tác vụ có giá trị virtual run time riêng, được kết hợp với một hệ số đặc biệt dựa trên độ ưu tiên.
 - Các tiến trình có độ ưu tiên bình thường có virtual run time tương đương với thời gian chạy thực tế.
 - Chọn tiến trình có virtual run time nhỏ nhất để thực thi tiếp.



8.1. Định thời trên Linux

Bộ định thời Real-time

- Định thời real-time dựa trên tiêu chuẩn POSIX.
 - Các tác vụ real-time có độ ưu tiên tĩnh.
- Độ ưu tiên được chia thành 2 phần: real-time (từ 0 đến 99) và normal (từ 100 đến 139).
- Giá trị nice -20 tương ứng với độ ưu tiên 100, +19 tương ứng với độ ưu tiên 139.

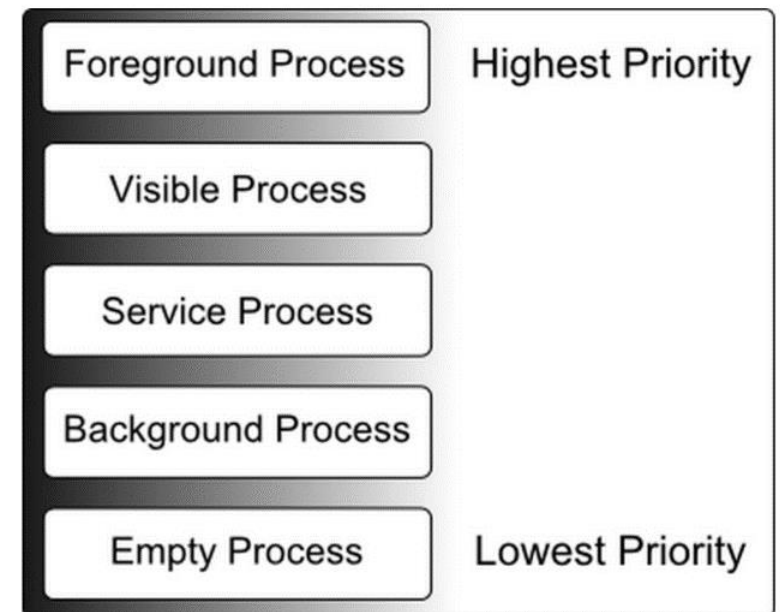




8.1. Định thời trên Linux

Định thời trên Android

- Sử dụng bộ định thời của Linux.
- Độ ưu tiên được phân chia theo nhóm của các tiến trình.
- Để thu hồi tài nguyên, Android có thể hủy (kill) các tiến trình dựa trên độ ưu tiên của chúng.





ĐỊNH THỜI TRÊN MỘT SỐ HỆ ĐIỀU HÀNH

8.2. Định thời trên Windows

08.



8.2. Định thời trên Windows

- Định thời theo độ ưu tiên với chế độ trưng dụng.
- Tác vụ có độ ưu tiên cao nhất luôn được chạy tiếp.
- Tiến trình sẽ được thực thi cho đến khi (1) block bởi system call, (2) hết quantum time, (3) bị thay thế bởi một tiến trình khác có độ ưu tiên cao hơn.
- Sử dụng 32 độ ưu tiên, được chia thành 2 lớp: variable (1-15) và real-time (16-31). Độ ưu tiên 0 dành cho quản lý bộ nhớ.
- Mỗi độ ưu tiên có hàng đợi riêng.
- Idle thread được chạy nếu không có bất cứ tác vụ nào trong hàng đợi.



8.2. Định thời trên Windows

- Các hàm thư viện Windows API cung cấp cho tiến trình các lớp ưu tiên sau:
 - REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS.
- Tiến trình có thể có các độ ưu tiên tương đối sau:
 - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, **NORMAL**, BELOW_NORMAL, LOWEST, IDLE



8.2. Định thời trên Windows

- Lớp ưu tiên và độ ưu tiên tương đối có thể kết hợp để xác định giá trị ưu tiên.
- Độ ưu tiên cơ sở (lúc khởi tạo) là **NORMAL** bên trong lớp.
- Khi hết quantum, độ ưu tiên có thể giảm nhưng không nhỏ hơn độ ưu tiên cơ sở.

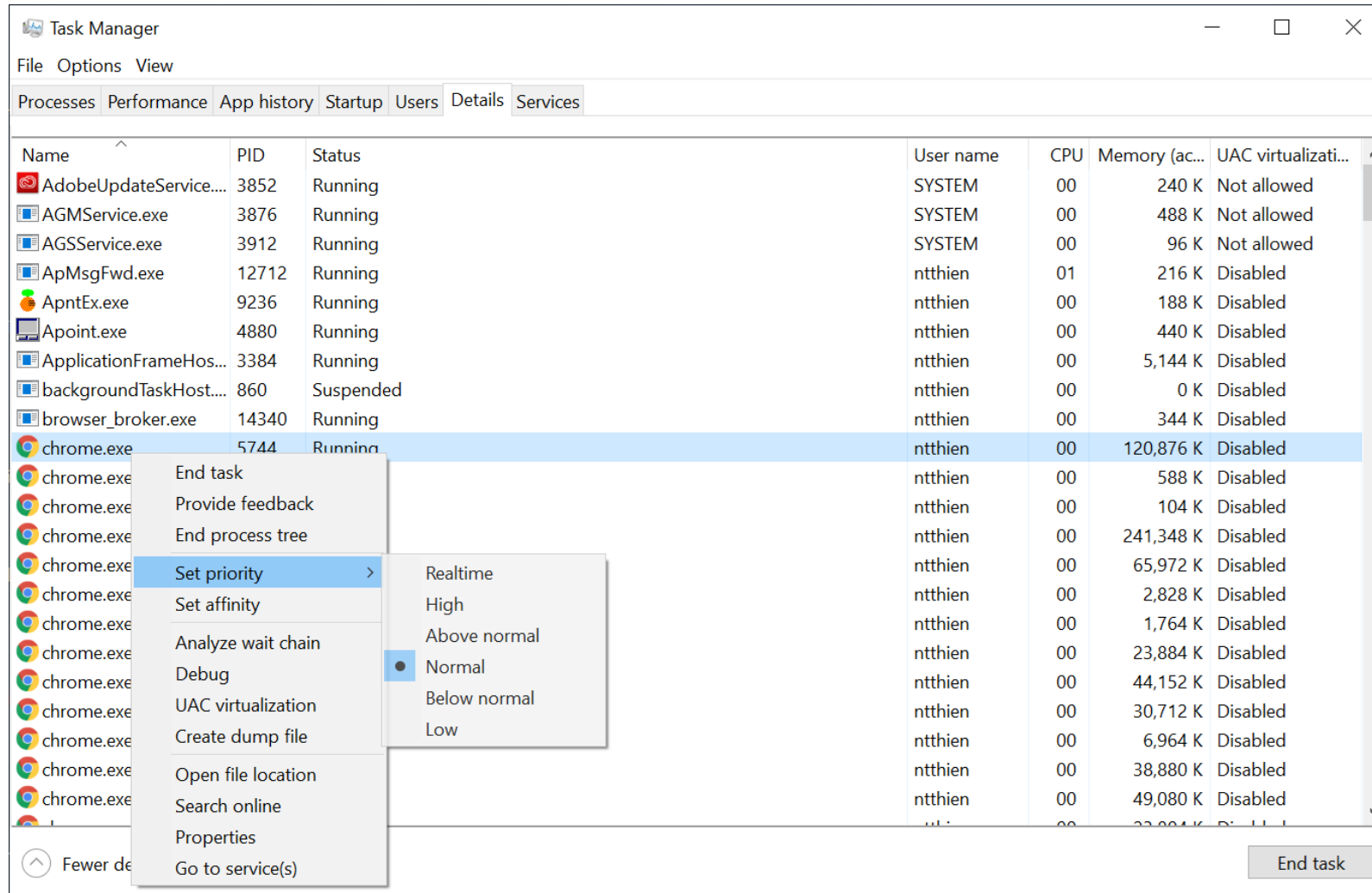
8.2. Định thời trên Windows

- Các độ ưu tiên trên Windows

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1



8.2. Định thời trên Windows





8.2. Định thời trên Windows

- Windows 7 có thêm user-mode scheduling (UMS):
 - Ứng dụng tạo và quản lý tiểu trình độc lập với nhân.
 - Hiệu quả hơn trong trường hợp có nhiều tiểu trình.
 - Định thời UMS được thực hiện với sự hỗ trợ của các thư viện như C++ Concurrency Runtime (ConcRT).



ĐỊNH THỜI TRÊN MỘT SỐ HỆ ĐIỀU HÀNH

8.3. Định thời trên Solaris (đọc thêm)

08.



8.3. Định thời trên Solaris

- Định thời theo độ ưu tiên.
- Có 6 lớp, mỗi lớp có độ ưu tiên khác nhau và giải thuật định thời khác nhau:
 - Time sharing (TS) – mặc định
 - Interactive (IA)
 - Real time (RT)
 - System (SYS)
 - Fair Share (FSS)
 - Fixed priority (FP)
- Lớp TS sử dụng giải thuật định thời MFQ.
- Độ ưu tiên càng lớn thì time slice càng nhỏ.



8.3. Định thời trên Solaris

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

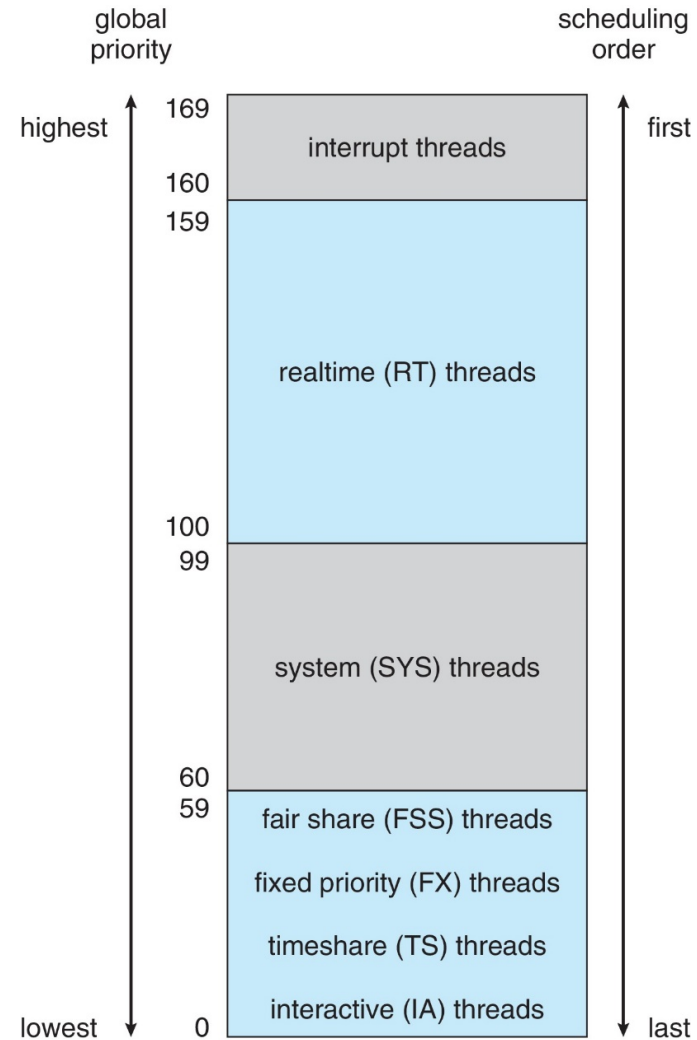


8.3. Định thời trên Solaris

- Bộ định thời chuyển đổi độ ưu tiên theo lớp thành độ ưu tiên toàn cục:
 - Tác vụ có độ ưu tiên cao nhất được chọn chạy tiếp.
 - Tiến trình sẽ được thực thi cho đến khi (1) block, (2) hết quantum time, (3) bị thay thế bởi một tiến trình khác có độ ưu tiên cao hơn.
 - Nếu có nhiều tiến trình có cùng độ ưu tiên, bộ định thời sẽ sử dụng hàng đợi round-robin.



8.3. Định thời trên Solaris





Tóm tắt nội dung buổi học

- Các giải thuật định thời: Highest Response Ratio Next (HRRN), Multilevel Queue, Multilevel Feedback Queue
- Định thời tiến trình (Thread scheduling)
- Định thời đa bộ xử lý (Multiple-processor scheduling)
- Định thời theo thời gian thực (Real-time CPU scheduling)
- Định thời trên một số hệ điều hành
 - Linux
 - Windows
 - Solaris



Câu hỏi ôn tập

- Mô tả và nêu ưu điểm, nhược điểm của từng giải thuật định thời: HRRN, MQ, MFQ.
- Định thời tiểu trình như thế nào?
- Có các cách tiếp cận nào để thực hiện định thời đa bộ xử lý? Ưu nhược điểm của từng cách tiếp cận?
- Cân bằng tải là gì? Tại sao phải cân bằng tải?
- Định thời theo thời gian thực như thế nào?
- Mô tả CFS?
- Trình bày đặc điểm của bộ định thời trên Windows?



THẢO LUẬN

